Serial Communication Protocols:
Inter-integrated Circuit

Nolan Gagnon

May 24, 2026

Overview

I^2C (or inter-integrated circuit) is a two-wire [1] serial communication protocol often used for communication between integrated circuits on the same printed circuit board. The two wires of the I^2C bus (*i.e.*, the physical connections that support I^2C communications) are called ***SDA*** (serial data line) and ***SCL*** (serial clock line), respectively. An I^2C bus supports multiple masters and numerous slaves. The number of slaves only becomes a problem once the combined capacitance of the slaves, masters, and bus wires becomes too high.

Communications with multiple slaves on the same I^2C bus are possible because each slave device has its own ***I^2C address*** that is unique on the bus. When the master (we will assume there is only one master for simplicity's sake) wants to send data to or receive data from a slave, it will transmit that slave's I^2C address on the bus. What happens next depends on the communication direction. If the master wants to send data to the slave, that data will be sent on the bus next. If, on the other hand, the master wants to receive data from the slave, the slave will transmit the requested data next.

Details

Protocol Overhead: When actually programming a device to use I^2C, there are many low-level details to consider. Most basic is the ***protocol overhead***, which refers to the extra signaling that must be done in order to successfully transmit data between two devices. The I^2C protocol has three non-data signals (sometimes referred to as conditions): (1) the **START** signal, (2) the **ACK** signal, and (3) the **STOP** signal. Depictions of each of these signals can be seen in Figure 1 below.

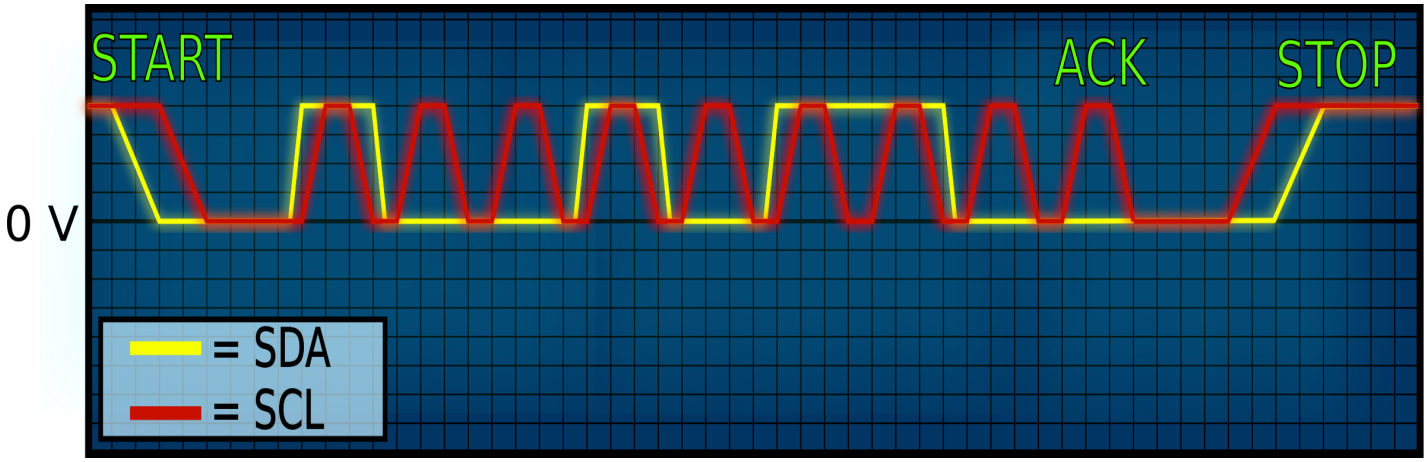

Figure 1: I^2C Overhead

A **START** condition occurs when the SDA signal (shown in yellow in Figure 1) transitions from a high voltage to a low voltage while the SCL signal (shown in red in Figure 1) is at a high voltage. This signal informs devices on the I^2C bus that they should start listening for transmissions from the master. The next section of signaling shown in the figure (after the **START** condition and before the **ACK** condition) corresponds to actual data transmission. In hexadecimal, the data being sent in this example is 0x96 (or 0b10010110 in binary). This is determined by looking at the voltage of the SDA line at each pulse of the SCL line. On the first pulse of SCL, SDA is high, so the corresponding data bit is a 1. On the second pulse of SCL, SDA is low, so the corresponding data bit is a 0. And so on and so forth. After the byte of data has been signaled

[1] The number two refers to the number of wires needed for communication between devices. Since any I^2C device must be powered when in use, it will also require two wires for GND and VCC.

on the I^2C bus, the intended receiving device will either acknowledge (**ACK** condition) that it received the data or it won't (**NACK** condition). Acknowledging the receipt of a data byte requires the receiver to pull the SDA line low during the ninth pulse of the current SCL cycle (note that the ninth SDA value is not part of the transmitted data). If the receiving device does not receive the data, then it does nothing, leaving the SDA line high during the ninth SCL pulse (we will explain later why the default value of SDA is high, instead of low). At the end of a data transmission, a **STOP** condition is signaled on the bus, indicating to the slave that the transmission is complete. As can be seen in Figure 1, a **STOP** condition occurs when SDA transitions from a low voltage to a high voltage, while SCL is at a high voltage.

Addressing: As mentioned previously, each device with an I^2C serial interface has an I^2C address. Quite often the addresses are 7 bits long, though they can sometimes have other lengths (10 bits is a common alternative).